Rockchip ROS2 Introduction

ID: RK-SM-YF-912

Release Version: V1.1.2

Release Date: 2026-04-11

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD.("ROCKCHIP")DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document describes how to compile ROS2 on the latest Rockchip Linux SDK (based on Buildroot). Unlike ROS2's officially supported systems such as Debian, we need to compile each ROS2 package from source.

Early versions of Rockchip Linux SDK included ROS1 compilation support, but it is now obsolete and no longer maintained. If needed, please refer to early SDK versions (before 2021).

Due to the rapid iteration of ROS2 and its strong dependency on Python versions, different releases of Rockchip Linux SDK only support the mainstream ROS2 versions available at the time of release.

The current ROS2 LTS versions are mainly Humble and Jazzy. Earlier versions such as Foxy and Iron have reached end-of-life. If needed, please refer to SDK versions released in 2023.

The Rockchip Linux SDK source code only provides the dependencies required by ROS2. ROS2 itself is compiled in a specific Docker environment. This design decouples ROS2 compilation from Rockchip Linux SDK as much as possible, facilitating ROS2 version updates and maintenance.

ROS2 documentation [1][2] shows that official Docker images are provided for cross-compiling arm/aarch64 architectures (e.g., Dockerfile_Ubuntu_arm64_prebuilt[2]). However, the target system only supports Ubuntu 18.04 (Bionic), which differs significantly from the rootfs in Rockchip Linux SDK. Drawing from this approach, we summarize ROS2 compilation into three steps:

- First, complete the Rockchip Linux SDK compilation, which includes dependencies required by ROS2 (such as Python3, bullet, opencv, eigen, etc.)
- Enter the Docker environment and cross-compile ROS2 using the Ubuntu environment along with Rockchip Linux SDK's cross-compilation toolchain and Sysroot
- Exit Docker and repackage the Rockchip Linux SDK rootfs

Product Versions

Chip Name	Buildroot Version Requirement in Rockchip Linux SDK
RK356x, RK3572, RK3576, RK3588, RV1126B and other arm64 chips	tag:linux-6.12-stan-rka0

Intended Audience

This document (guide) is primarily intended for the following engineers:

- Technical Support Engineers
- Software Development Engineers

Revision History

Version	Author	Date	Description
V1.0.0	zhengsq	2021-09-09	Initial version
V1.1.0	zhengsq	2023-09-21	Updated to match current new Rockchip Linux SDK, added ROS2 humble and iron support, modified compilation Docker environment, adjusted document structure
V1.1.1	zhengsq	2024-01-11	Minor text corrections
V1.1.2	zhengsq	2026-04-11	Added ROS2 Jazzy/humble support on latest SDK, marked earlier versions as obsolete

Table of Contents

Rockchip ROS2 Introduction

1. ROS2 Versions
2. Compilation Environment
3. Compiling ROS2 Dependencies
4. Host/Docker/Source Preparation
    4.1 Downloading ROS2 Source Code
    4.2 Applying Patches
    4.3 Copying Scripts and Configuration
    4.4 Building Docker Image
5. Entering Docker Image to Compile ROS2
    5.1 Compiling Individual ROS2 Packages and Applications
    5.2 Packaging rootfs and Running ROS2
6. Common Issues
    6.1 Manual TRY_RUN Handling Required for Cross-Compiling Humble
    6.2 Packages Requiring UI Display Are Not Supported
7. References

1. ROS2 Versions

The following 2 LTS versions are supported:

ROS2 Release	Detailed Version	Link
jazzy	ROS 2 Jazzy Jalisco - Patch Release 7	https://github.com/ros2/ros2/releases/tag/release-jazzy-20260128
humble	ROS 2 Humble Hawksbill - Patch Release 13	https://github.com/ros2/ros2/releases/tag/release-humble-20250721

- Note: Other ROS2 versions not listed (such as Foxy) and ROS1 versions have not been verified for compilation.

2. Compilation Environment

Recommended Host:

- Ubuntu 20.04 or later
- Follow other Rockchip Linux SDK compilation requirements; compile ROS2 on the same machine
- Network access to GitHub is required, as a large number of source packages need to be downloaded from GitHub

Rockchip Linux SDK provides unified ROS2 cross-compilation scripts and patches in the `docs/Patches/ROS2/` directory. The directory structure is as follows:

```
ROS2/
├── build_ros2.sh           # Unified build script (supports humble/jazzy parameter)
├── rosdep.Dockerfile        # Unified Dockerfile (supports --build-arg ROS_DISTRO)
├── cross-compile/
│   ├── cross-compile.mixin  # CMake cross-compilation config (shared by both versions)
│   └── index.yaml           # colcon mixin index file
├── patches/
│   ├── humble/              # ROS2 Humble patches
│   │   ├── ament_cmake/
│   │   │   └── 0001-add-CMAKE_SYSTEM_PROCESSOR-support-for-cross-compila.patch
│   │   └── mimick_vendor/
│   │       └── 0001-Set-cross-compiling-CMAKE_SYSTEM_NAME-and-CMAKE_SYST.patch
│   └── jazzy/               # ROS2 Jazzy patches
│       └── ament_cmake/
│           └── 0001-add-CMAKE_SYSTEM_PROCESSOR-support-for-cross-compila.patch
```

Supported ROS2 versions and patch details:

Version	Release Date	Patches	Description
Humble	2022-05 (LTS)	2	Requires ament_cmake + mimick_vendor patches
Jazzy	2024-05 (LTS)	1	Only requires ament_cmake patch

In the Buildroot directory of Rockchip Linux SDK, check if the corresponding ROS2 dependency config files exist:

```
user@host:~/linux-sdk/buildroot$ ls -1 configs/rockchip/ros2/
configs/rockchip/ros2/jazzy_dep.config
configs/rockchip/ros2/humble_dep.config
```

- If these files do not exist, please update to a newer SDK version.

3. Compiling ROS2 Dependencies

In the Buildroot project of Rockchip Linux SDK, the configuration files under `configs/rockchip/ros2/` provide the dependencies required for compiling and running ROS2. These need to be added and compiled into the rootfs. For example, add the corresponding version config (humble_dep.config or jazzy_dep.config) to rockchip_rk3572_defconfig:

```
user@host:~/linux-sdk/buildroot$ git diff configs/rockchip_rk3572_defconfig
diff --git a/configs/rockchip_rk3572_defconfig b/configs/rockchip_rk3572_defconfig
index 8dc0afcd46..7072049b5c 100644
--- a/configs/rockchip_rk3572_defconfig
+++ b/configs/rockchip_rk3572_defconfig
@@ -22,3 +22,4 @@
 #include "network/chromium.config"
 #include "ai/npu2.config"
 #include "gui/weston.config"
+#include "ros2/jazzy_dep.config"
```

After completing the full rootfs compilation, proceed to the next step.

4. Host/Docker/Source Preparation

Install Docker on the Ubuntu PC:

```
sudo apt install docker.io
sudo usermod -aG docker $USER
newgrp docker  # Log in to the docker group

sudo apt install python3-vcstool # Install vcs-import tool
```

Install qemu-user-binfmt (register binfmt rules):

```
sudo apt install -y qemu-user-binfmt

# Verify installation (should see qemu-aarch64 and other registered items)
ls /proc/sys/fs/binfmt_misc/qemu-*
/proc/sys/fs/binfmt_misc/qemu-aarch64
/proc/sys/fs/binfmt_misc/qemu-arm
...
```

binfmt Function Description:

- binfmt_misc is a Linux kernel feature that allows automatic recognition of ELF binaries for non-native architectures
- After registration, the kernel will automatically call `/usr/bin/qemu-aarch64-static` when executing aarch64 binaries
- **Note**: binfmt only starts QEMU; you also need to configure `QEMU_LD_PREFIX` to tell QEMU where to find the target system's libraries

4.1 Downloading ROS2 Source Code

```
# Create working directory
mkdir -p ~/ros2 && cd ~/ros2

# Download and extract ROS2 release package
# Humble:
wget https://github.com/ros2/ros2/archive/refs/tags/release-humble-20250721.tar.gz
tar zxf release-humble-20250721.tar.gz
# Jazzy:
wget https://github.com/ros2/ros2/archive/refs/tags/release-jazzy-20260128.tar.gz
tar zxf release-jazzy-20260128.tar.gz

# Use vcs-import to import source code
mkdir src
# For Humble version:
vcs-import -w 10 --retry 10 --skip-existing --recursive src < ros2-release-humble-
20250721/ros2.repos
# For Jazzy version:
vcs-import -w 10 --retry 10 --skip-existing --recursive src < ros2-release-jazzy-
20260128/ros2.repos
```

- During docker build, the src source code will be copied. The src directory must be in the same directory as rosdep.Dockerfile.

4.2 Applying Patches

Use the `git am` command to apply the corresponding patches. Patches are located in the SDK's `docs/Patches/ROS2/patches/` directory, organized by version:

```
SDK_PATCHES=~/rk/linux-sdk/docs/Patches/ROS2/patches

# Patches for Humble (2 patches)
cp $SDK_PATCHES/humble/ament_cmake/*.patch src/ament/ament_cmake/
cp $SDK_PATCHES/humble/mimick_vendor/*.patch src/ros2/mimick_vendor/
cd src/ament/ament_cmake && git am *.patch && cd -
cd src/ros2/mimick_vendor && git am *.patch && cd -

# Patches for Jazzy (1 patch)
cp $SDK_PATCHES/jazzy/ament_cmake/*.patch src/ament/ament_cmake/
cd src/ament/ament_cmake && git am *.patch && cd -
```

Patch Description:

Patch	Versions	Purpose	Apply Location
ament_cmake	humble + jazzy	Add `CMAKE_SYSTEM_PROCESSOR` cross-compilation support, enabling third-party libraries (such as googletest) to correctly identify the target architecture	`src/ament/ament_cmake/`
mimick_vendor	humble only	Set cross-compiling `CMAKE_SYSTEM_NAME` and `CMAKE_SYSTEM_PROCESSOR` to prevent mimick from attempting to compile HOST architecture test programs	`src/ros2/mimick_vendor/`

- Note: Jazzy does not require the mimick_vendor patch because the upstream has already fixed the related issue.

4.3 Copying Scripts and Configuration

Copy the unified build script, Dockerfile, and cross-compilation configuration provided by the SDK to the working directory:

```
SDK_ROS2=~/rk/linux-sdk/docs/Patches/ROS2

cp $SDK_ROS2/build_ros2.sh .
cp $SDK_ROS2/rosdep.Dockerfile .
cp -r $SDK_ROS2/cross-compile .
chmod +x build_ros2.sh
```

4.4 Building Docker Image

Create a Docker image using the unified `rosdep.Dockerfile` provided by Rockchip Linux SDK. Specify the ROS2 version via the `--build-arg ROS_DISTRO` parameter:

```
# Humble:
docker build --build-arg ROS_DISTRO=humble \
    --build-arg HOST_USER=$(whoami) --build-arg HOST_UID=$(id -u) --build-arg HOST_GID=$(id -g) \
    -t humble-ros2-builder -f rosdep.Dockerfile ./

# Jazzy:
docker build --build-arg ROS_DISTRO=jazzy \
    --build-arg HOST_USER=$(whoami) --build-arg HOST_UID=$(id -u) --build-arg HOST_GID=$(id -g) \
    -t jazzy-ros2-builder -f rosdep.Dockerfile ./
```

- The Docker image uses the same user/group as the host to maintain consistent file access permissions between host and Docker.
- Default password: rockchip

5. Entering Docker Image to Compile ROS2

Bind the compiled Rockchip Linux SDK to the Docker environment and execute the compilation:

```
cd ~/ros2

# Define variables (modify according to your actual situation)
export SDK_PATH=~/rk/linux-sdk
export BOARD_CONFIG=rockchip_rk3572_evb1_robot
export OUTPUT_DIR=${SDK_PATH}/buildroot/output/${BOARD_CONFIG}
export ROS_DISTRO=jazzy  # or humble
```

Run with Current User Permissions (Recommended to Avoid Permission Issues)

```
# Enter the Docker image, maintaining Host user permissions
# Use --privileged flag to allow the container to access the host's binfmt
docker run -it --rm --privileged \
  -u $(id -u):$(id -g) \
  -v /etc/passwd:/etc/passwd:ro \
  -v /etc/group:/etc/group:ro \
  --mount type=bind,source=${OUTPUT_DIR},target=/buildroot \
  ${ROS_DISTRO}-ros2-builder:latest
```

Compilation Execution:

```
user@host:~$ ls
build_ros2.sh  src

# Execute compilation (supports ROS_DISTRO parameter or environment variable)
user@host:~$ ./build_ros2.sh jazzy
# Or
user@host:~$ ROS_DISTRO=jazzy ./build_ros2.sh
....
Summary: 328 packages finished [1h 2min 16s]
....
```

Parameter Description:

- `source=`: Must be changed to the **absolute path** of the corresponding Rockchip Linux SDK compilation output directory. This path should contain target, host, staging, build, and other directories.
- Compiled target files are located in the Rockchip Linux SDK corresponding Image directory: `${OUTPUT_DIR}/target/opt/ros`
- Compilation intermediate files are stored in the `${OUTPUT_DIR}/build/ros` directory

Compilation Time Reference:

- Full compilation: approximately 1-2 hours (depends on Host performance; reference configuration: 16-core CPU, 32GB RAM, SSD)
- Incremental compilation: approximately 5-10 minutes (only modifying a few packages)

If build_ros2.sh completes without errors, the compilation is successful. However, some packages cannot be compiled or executed in the Buildroot SDK environment, such as:

- rviz: depends on X11/desktop. If you need this functionality, use the Ubuntu arm image directly instead of Buildroot
- turtlesim: depends on UI display
- If you want to exclude a package from compilation, create a COLCON_IGNORE file in the corresponding src path. For example: `touch src/ros/ros_tutorials/turtlesim/COLCON_IGNORE`

5.1 Compiling Individual ROS2 Packages and Applications

Refer to the `build_ros2.sh` script. Use the colcon build parameter `--packages-select <package_name>` to compile individual packages.

5.2 Packaging rootfs and Running ROS2

After completing the full ROS2 compilation above, enter Rockchip Linux SDK and repackage the rootfs. ROS2 is installed under the /opt/ros directory.

```
cd ~/rk/linux-sdk
./build.sh rootfs   # Repackage rootfs.img
```

After flashing rootfs.img, enter the board and execute the Hello World Demo:

```
cd /opt/ros/
export COLCON_CURRENT_PREFIX=/opt/ros
export ROS_HOME=/userdata/
source ./local_setup.sh
ros2 pkg list
ros2 pkg executables

ros2 run demo_nodes_cpp listener &
ros2 run demo_nodes_cpp talker
[INFO] [1501839280.834017748] [talker]: Publishing: 'Hello World: 1'
[INFO] [1501839280.839280957] [listener]: I heard: [Hello World: 1]
[INFO] [1501839281.831636015] [talker]: Publishing: 'Hello World: 2']
[INFO] [1501839281.835092640] [listener]: I heard: [Hello World: 2]
[INFO] [1501839282.831618532] [talker]: Publishing: 'Hello World: 3']
[INFO] [1501839282.835336782] [listener]: I heard: [Hello World: 3]

ros2 run demo_nodes_py listener &
ros2 run demo_nodes_py talker
```

6. Common Issues

6.1 Manual TRY_RUN Handling Required for Cross-Compiling Humble

fastrtps TRY_RUN prompt:

```
--- stderr: fastrtps
CMake Error: TRY_RUN() invoked in cross-compiling mode, please set the following cache
variables appropriately:
   SM_RUN_RESULT (advanced)
   SM_RUN_RESULT__TRYRUN_OUTPUT (advanced)
For details see /buildroot/build/ros/fastrtps/TryRunResults.cmake
```

- Follow the instructions to run the application on the board and fill in the results accordingly. For example:

```
user@host:~$ /tmp/cmTC_4f573-SM_RUN_RESULT
PTHREAD_RWLOCK_PREFER_READER_NP

# Based on the execution results above, fill in the results in Docker:
vim /buildroot/build/ros/fastrtps/TryRunResults.cmake
  ....
  set( SM_RUN_RESULT
       "0"
       CACHE STRING "PTHREAD_RWLOCK_PREFER_READER_NP" FORCE)

  set( SM_RUN_RESULT__TRYRUN_OUTPUT
       "0"
       CACHE STRING "PTHREAD_RWLOCK_PREFER_READER_NP" FORCE)
```

6.2 Packages Requiring UI Display Are Not Supported

Rockchip Linux SDK does not support QT, so the following QT-dependent packages cannot be compiled or run:

- rviz-related packages
- ros-visualization-related packages, such as rqt, python_qt_binding, qt_gui_core
- turtlesim

7. References

1. https://docs.ros.org/en/jazzy/Installation/Alternatives/Latest-Development-Setup.html
2. https://github.com/ros-tooling/cross_compile.git